

Project Documentation: Production-Grade Local RAG System

1. Project Overview

This project establishes a **Retrieval-Augmented Generation (RAG)** pipeline designed for high-performance document intelligence on a local machine. By leveraging **Groq's LPU (Language Processing Unit)** technology for inference and **FAISS** for vector indexing, the system provides near-instantaneous responses to natural language queries based on private PDF datasets. This "production-grade" architecture prioritizes data privacy, modularity, and cost-efficiency by utilizing open-source models and free-tier APIs.

2. System Architecture

The system follows a classic four-stage RAG pipeline:

1. **Data Ingestion:** Utilizing PyPDFLoader to extract raw text and metadata from PDF files.
 2. **Semantic Chunking:** Documents are processed via RecursiveCharacterTextSplitter, which preserves context by splitting text at logical boundaries (paragraphs and sentences) rather than arbitrary character counts.
 3. **Vectorization:** Text chunks are converted into 384-dimensional dense vectors using the BAAI/bge-small-en-v1.5 embedding model.
 4. **Storage & Retrieval:** Vectors are stored in a local **FAISS (Facebook AI Similarity Search)** index. At query time, the system performs a similarity search to retrieve the "Top-K" most relevant chunks.
 5. **Augmentation & Generation:** The retrieved context is injected into a specialized prompt and sent to the **Llama-3.3-70b** model on Groq for final answer synthesis.
-

3. Technical Stack

- **Orchestration:** LangChain (Python)
 - **LLM Engine:** Groq Cloud API (llama-3.3-70b-versatile)
 - **Embeddings:** Hugging Face Transformers (Local CPU execution)
 - **Vector Store:** FAISS (FlatL2 Index)
 - **Environment Management:** python-dotenv for API security
-

4. Setup and Installation

Prerequisites: Python 3.9+, Groq API Key.

Installation:

Bash

```
pip install groq faiss-cpu sentence-transformers langchain-huggingface langchain-groq pypdf python-dotenv
```

Configuration:

Create a .env file in the root directory:

Code snippet

```
GROQ_API_KEY=your_key_here
```

```
GROQ_MODEL=llama-3.3-70b-versatile
```

```
EMBEDDING_MODEL=BAAI/bge-small-en-v1.5
```

5. Implementation Details

The core logic is encapsulated in the ProductionRAG class. Key features include:

- **Persistence:** The FAISS index is saved to a local directory (faiss_index/), allowing the tool to be restarted without re-processing documents.
 - **Source Attribution:** The system returns not just the answer, but the specific page numbers and text snippets used, ensuring transparency and reducing LLM hallucinations.
 - **Memory Efficiency:** By running embeddings locally on the CPU, the system avoids expensive cloud embedding costs while maintaining low latency.
-

6. Production Considerations & Scalability

While currently a local tool, this architecture is built for scaling:

- **API Transition:** The class-based structure allows for easy integration into a **FastAPI** or **Flask** wrapper for deployment as a microservice.
- **Security:** API keys are never hardcoded, following standard DevOps security protocols.
- **Reliability:** The use of temperature=0 ensures deterministic, factual outputs suitable for enterprise environments.

7. Conclusion

This RAG implementation demonstrates that a high-performance AI assistant can be built without a massive infrastructure budget. By combining the speed of Groq with the reliability of local vector storage, the project provides a robust foundation for building advanced document search and analysis tools.